

CS6530 – APPLIED CRYPTOGRAPHY

Assignment 2

Authenticated Ephemeral Key Establishment

Technical Report

Name	Nipun Bargal
Roll Number	MD24B033
Partner	SANTHOSH D R (CS26E004)
Date	September 18, 2026

1. Introduction

In this assignment, We implemented an authenticated ephemeral key establishment protocol between Alice and Bob. The communication is done through an untrusted network. The main purpose of the assignment is to establish fresh session keys and then use those keys to send protected messages. The implementation uses X25519, Ed25519, SHA-256, HKDF-SHA-256 and AES-256-GCM as given in the assignment.

I also tested the protocol under normal conditions and under different attacks. The tests were done for a normal authenticated session, a man-in-the-middle attack, a replay attack and a forward secrecy experiment.

2. Cryptographic Algorithms Used

Algorithm	Purpose
X25519	Used to create the fresh shared secret between Alice and Bob.
Ed25519	Used for long-term identity authentication and signatures.
SHA-256	Used to create the handshake transcript hash.
HKDF-SHA-256	Used to derive the two session traffic keys.
AES-256-GCM	Used to encrypt and authenticate application messages.

3. Handshake Process

The handshake is completed using four messages, M1 to M4. Alice first creates a new session ID and a new ephemeral X25519 key pair. She sends the session ID and ephemeral public key to Bob in M1.

Bob then creates his own session ID and ephemeral X25519 key pair. In M2 he sends his session ID and ephemeral public key back to Alice. He also sends Alice's session ID back so Alice can check that the correct session is being used.

After M2, both sides create the same canonical transcript. The transcript contains the protocol ID, both roll-number IDs, both session IDs and both ephemeral public keys. The transcript is 124 bytes long. SHA-256 is then used to create the Transcript_Hash.

Alice signs the Transcript_Hash using her long-term Ed25519 private key and sends the signature in M3. Bob verifies the signature using Alice's trusted public key. Bob then signs the same Transcript_Hash and sends the signature in M4. Alice verifies it using Bob's trusted public key. If the SID is wrong or a signature fails, the session is stopped.

4. Key Derivation

After the handshake, both Alice and Bob calculate the X25519 shared secret using their ephemeral private key and the peer's ephemeral public key. The raw X25519 shared secret is not used directly as an AES key.

HKDF-SHA-256 is used with the Transcript_Hash as the salt. Two different information strings are used so that two different 32-byte keys are created. One key is used from Alice to Bob and the other key is used from Bob to Alice.

The derivation is:

$$\text{PRK} = \text{HKDF-Extract}(\text{salt} = \text{Transcript_Hash}, \text{IKM} = \text{Shared_Secret})$$
$$\text{K_Alice_to_Bob} = \text{HKDF-Expand}(\text{PRK}, \text{info} = \text{"CS6530-A2 Alice->Bob"}, \text{L} = 32)$$
$$\text{K_Bob_to_Alice} = \text{HKDF-Expand}(\text{PRK}, \text{info} = \text{"CS6530-A2 Bob->Alice"}, \text{L} = 32)$$

5. AES-256-GCM Communication

For application messages, AES-256-GCM is used. Each direction has its own counter and the counter starts from 0. The nonce is created using 0x00000000 followed by the 64-bit counter in big-endian format.

The AAD contains the sender ID, receiver ID, Alice session ID, Bob session ID and message counter. The AAD is not encrypted, but it is included in the GCM authentication calculation. This means that if someone changes the AAD, the GCM verification will fail.

The receiver only accepts the next expected counter. Because of this, an old record cannot simply be sent again and accepted as a new message.

6. Connectivity Test

Before testing the cryptographic protocol, I checked the supplied TCP connectivity program. The local test completed successfully and the messages were received on both sides.

Alice output:

Alice: connected to Bob

Alice received: Hello Alice

Bob output:

Bob: connected to ('127.0.0.1', 36050)

Bob received: Hello Bob

This confirms that the basic transport and message framing are working. For the final submission, the same test should also be shown with Alice and Bob running on the two physical computers required by the assignment.

7. TR-1 – Normal Authenticated Session

The normal session test completed the M1-M4 handshake successfully. Both sides generated the same X25519 shared secret and derived the same directional traffic keys using HKDF.

Application messages were then exchanged in both directions. The counters used were 0, 1 and 2. The messages were successfully verified and decrypted using AES-256-GCM.

The evidence also shows that the raw X25519 shared secret and the derived traffic keys have different fingerprints. This confirms that the shared secret is not being used directly as the AES key.

8. TR-2 – Man-in-the-Middle Attack

For the MITM experiment, I first ran the protocol with Ed25519 authentication disabled. In this mode Mallory can replace the ephemeral public keys sent between Alice and Bob. This gives Mallory one shared secret with Alice and another shared secret with Bob.

Mallory was then able to intercept a message, modify it and forward it to Bob. For example, the original message was changed to include '[intercepted by Mallory]'. This shows why the ephemeral keys need to be authenticated.

After restoring Ed25519 authentication, the same substitution caused the transcript hashes to be different. Bob could not verify Alice's signature because the signed transcript did not match the modified transcript. The handshake was aborted and no application data was accepted.

Observed authenticated-mode result: hashes_differ = true, followed by signature verification failure and session abort.

9. TR-3 – Replay Attack

For the replay test, a valid application record with counter 0 was captured. The first time it was received, it was accepted. After that, the receiver expected the next counter value. When the same counter-0 record was sent again, it was rejected as stale or unexpected.

Observed result:

counter 0 → ACCEPT

counter 1 → ACCEPT

replayed counter 0 → REJECT

The replayed record can still have a valid GCM tag because the record itself has not been changed. GCM checks whether the record is authentic and has not been modified, but it does not keep a history of previously accepted records. Therefore the counter state is needed for replay protection.

10. TR-4 – Forward Secrecy

For the forward secrecy experiment, a normal session was recorded first. The idea was to keep the old encrypted traffic and then simulate a later compromise of Alice's long-term Ed25519 private key.

The long-term Ed25519 private key could not be used to reconstruct the old X25519 shared secret or the old traffic key. This is because the old ephemeral X25519 private key had been discarded.

As a controlled comparison, when the old ephemeral X25519 private key was kept, the old traffic key could be reconstructed and the recorded plaintext could be recovered. The fingerprints matched:

Original key fingerprint: 7f28efdfc4c78bd3

Reconstructed key fingerprint: 7f28efdfc4c78bd3

keys_match: true

plaintext_recovered: S1 confidential record

11. Results

Test	Result	Status
Connectivity test	Alice and Bob exchanged messages	Completed
TR-1 Normal session	Authenticated and encrypted communication worked	Completed
TR-2 MITM weakened mode	Mallory modified intercepted traffic	Completed
TR-2 MITM authenticated mode	Signature verification failed and session stopped	Completed
TR-3 Replay	Old counter was rejected	Completed
TR-4 Forward secrecy	Old session could not be reconstructed from long-term key alone	Completed

12. Security Discussion

The main security idea in this protocol is that X25519 provides fresh temporary key agreement while Ed25519 provides authentication. The signatures cover the complete handshake transcript, including the ephemeral public keys. Therefore an attacker changing an ephemeral public key also changes the transcript that has to be verified.

The two HKDF information strings make the Alice-to-Bob and Bob-to-Alice traffic keys separate. AES-GCM then protects the application messages. The counters provide an additional freshness check and stop an already accepted message from being accepted again.

The forward secrecy test shows why the ephemeral X25519 private keys must be deleted after a session. A later compromise of the long-term authentication key does not by itself reveal the old session key.

13. Conclusion

In this assignment, I implemented and tested an authenticated ephemeral key establishment protocol. The normal test showed that Alice and Bob can establish the same session keys and communicate using AES-256-GCM. The MITM test showed that removing authentication allows an attacker to modify the communication, while restoring authentication causes the handshake to fail. The replay test showed that stale counters are rejected. The forward secrecy test showed that the old session cannot be reconstructed using only the later compromised Ed25519 private key.

The remaining work for the final evidence is to include the actual two-machine connectivity screenshot/log and, if the bonus is required, a Wireshark packet capture with the M1-M4 and application records identified.